Configuration targets three levels: the model, types, and properties. You can configure the schema EF Core uses for objects at the model level. Type configuration options enable you to configure the table's name that a type maps to or how relationships should be specified between types. Property configuration provides extensive options for managing how individual properties map to columns, including their name, data type, default values, and so on.

Configuration can be applied in two ways: by decorating classes and properties with attributes or by using a fluent API that consists of a collection of extension methods that can be chained together. Attributes only provide a subset of configuration options. Therefore, for any reasonably complex model, you are likely to need to rely on the fluent API for some configuration. Consequently, it makes sense to use the fluent API for *all* configurations, keeping your configuration code consistent, and therefore easier to reason about, and in one place.

So where should you put your fluent API configuration code? You have two options: you can override the `DbContext OnModelCreating` method in your own context class and place the configuration code there, or you can place configuration code in separate classes for each entity and then reference those classes in the `OnModelCreating` method. You will adopt the latter approach, since it is the recommended way to manage this aspect of your application.

Configuration classes derive from `IEntityTypeConfiguration<TEntity>`, which implements one method, `Configure`, which takes an `EntityTypeBuilder<TEntity>` as a parameter. It is in this method that you will place your configuration code.